Oracle Utilities Framework v25.10
Product Fix Design
Bug 38627909

Last Modified: February 19, 2026 10:33 PM
Printed On:

Oracle Utilities Framework Product Fix Design

Copyright 2026 by Oracle. All rights reserved.

The information contained in this document is proprietary and confidential to Oracle and is subject to the non-disclosure obligations described in the license agreement between your company and Oracle. You may not provide this document or any copy of this document to third parties outside your company unless you first obtain the written permission of Oracle.

This document introduces new and changed business and technical features that Oracle has included in the"Single Fix" described in this document.

The functionality of the Single Fix described in this document may be superseded by functionality included in a future Single Fix, Release or Update. Oracle reserves the right to revise or expand the functionality of any future Single Fix, Release or Update of Framework in a manner compatible with Oracle's business objectives, and the business objectives of each of Oracle's customers, at the time that Oracle makes the software code generally commercially available.

Oracle will provide you with additional information concerning product Releases or Updates after Oracle finalizes the related specifications and tests the product component(s). Final documentation will be shipped to you with the media containing the Release or Update.

Please contact Oracle Customer Service Group if you have any questions or comments concerning this document, which has been prepared for your benefit as a licensee of Framework.

 Copyright ©2026, Oracle. All rights reserved.

Table of Contents

Table of Contents 3

Reported Issue 3

Solution Design 4

Reported Issue

Contents

Description
Steps To Recreate

Description

Users are unable to use the concatenateGroovyStrings method to build string literals that include dynamic integer index values. One use case is generating bind variable names for primary keys, where an integer from 1 to 5 can be appended based on how many primary key columns a table has.

Copyright ©2026, Oracle. All rights reserved.

Steps To Recreate

Create a new Groovy script that builds a query to fetch records from the database. The script should constructs the SQL using the concatenateGroovyStrings method and generate bind variable names dynamically based on the number of primary key columns in the table.

Determine the number of primary key columns, then use a for loop to append integer index value to column names and bind variables using concatenateGroovyStrings. When you click Save, the application throws an error indicating that integers are not allowed as arguments to concatenateGroovyStrings.

Example:

```
String sql = concatenateGroovyStrings("SELECT * FROM F1_MO_UPD WHERE ")
for (int i = 1; i <= pkCount; i++) {
   sql = concatenateGroovyStrings(sql, "PK_VALUE", i, " = ", ":pkVal", i)
   if (i < pkCount) {
      sql = concatenateGroovyStrings(sql, " AND ")
   }
}
def query = createPreparedStatement(sql, "comment")
```

Solution Design

Contents

Summary
FW Changes

Summary

Change Type	Y/N
Does this fix require any Admin (Soft) Table Changes to be made by the client?	N
Does this fix require upgrade data changes to be applied?	N
Does the fix involve DB Changes?	N
Is there an impact on any user interface or any other on-line behavior?	N
Is there an impact on any batch process?	N
Is there an impact on CM – Custom Modification processing?	N

The following questions are only applicable to products built on Oracle Utilities Application Framework version 22A / 4.5.0.0.0 or earlier.

Change Type	Y/N
Does this fix require an application viewer update for Algorithm data (run F1-	N

 Copyright ©2026, Oracle. All rights reserved.

AVALG)?	
Does this fix require an application viewer update for MO data (run F1-AVMO)?	N
Does this fix require an application viewer update for Table data (run F1-AVTBL)?	N

FW Changes

The framework has added several utility methods that allow integers to be concatenated with strings, and then used as string literals in concatenateGroovyStrings. The new methods are:

```
String concatenateBindName(String bindBase, int index);
String concatenateSqlInClause(String columnExpr, String bindBase, int start, int count);
String literalFor(int i);
String literalFor(long l);
```

Examples:

```
concatenateBindName(":pkValue", 1);                          // ":pkValue1"
concatenateSqlInClause("PK_VALUE", ":pkValue", 1, 3); // "PK_VALUE IN (:pkValue1, :pkValue2,
:pkValue3)"
literalFor(5);                                               // "5"
```

Recommended Test Cases

Create a new Groovy script that builds a query to fetch records from the database. The script should construct the SQL using the concatenateGroovyStrings method and generate bind variable names dynamically based on the number of primary key columns in the table.

Determine the number of primary key columns, then use a for loop to append the integer index value to column names and bind variables using the new helper methods literalFor and concatenateBindName, respectively. These methods can be used in conjunction with concatenateGroovyStrings. When you click Save, the script should save successfully.

Example:

```
String sql = concatenateGroovyStrings("SELECT * FROM F1_MO_UPD WHERE ")
for (int i = 1; i <= pkCount; i++) {
   sql = concatenateGroovyStrings(sql, "PK_VALUE", literalFor(i), " = ",
concatenateBindName(":pkVal", i))
   if (i < pkCount) {
      sql = concatenateGroovyStrings(sql, " AND ")
   }
}
def query = createPreparedStatement(sql, "comment")
```

sql output (assuming pkCount = 3):

Copyright ©2026, Oracle. All rights reserved.

```
SELECT * FROM F1_MO_UPD WHERE PK_VALUE1 = :pkVal1 AND PK_VALUE2 = :pkVal2
AND PK_VALUE3 = :pkVal3
```

 Copyright ©2026, Oracle. All rights reserved.